

CAPA / IDENTIFICAÇÃO

- **Instituição:** Universidade de Aveiro — DETI
- **Unidade Curricular:** Métodos Probabilísticos para Engenharia Informática (MPEI 2025-2026)
- **Trabalho Prático:** Feed de Notícias Inteligente e Personalizado
- **Autores:** David Monteiro (125793) e Murilo Frezzato (125487)

1. INTRODUÇÃO E OBJETIVO GERAL

O presente relatório descreve o desenvolvimento, teste e validação de um ecossistema computacional modular assente em estruturas de dados e algoritmos probabilísticos. O objetivo primordial consistiu no desenho e implementação autónoma de três componentes fundamentais em Matlab: um Filtro de Bloom, um Classificador Naïve Bayes e um Estimador de Similaridade MinHash.

Para demonstrar o uso conjunto e integrado destes algoritmos, rejeitou-se a replicação do exemplo genérico de deteção de spam em e-mails contido no guião pedagógico, visando garantir a originalidade e a ambição do projeto. Em sua substituição, desenvolveu-se um **Feed de Notícias Inteligente e Personalizado** utilizando dados reais do dataset de notícias `news.csv`. Este sistema opera como um pipeline em tempo real focado em três frentes: controlo de redundância para o utilizador, categorização temática automatizada e agrupamento dinâmico de artigos similares vindos de órgãos de comunicação concorrentes.

2. ARQUITETURA E FUNDAMENTAÇÃO DOS MÓDULOS

2.1. Bloom Filter (*bloomFilter.m*)

Desenvolvido como uma classe parametrizável, o módulo suporta os modos clássico (vetor binário) e *counting* (vetor de contadores de tipo inteiro de 16 bits). O dimensionamento do tamanho do filtro em bits (variável `numBits`) e o número ideal de funções de dispersão (variável `numHashes`) seguem o rigor teórico das probabilidades:

- O cálculo do número de bits necessário baseia-se no número esperado de itens e na taxa alvo de falsos positivos. Em código Matlab, a fórmula exata é calculada como:
`obj.numBits = ceil((-expectedItems * log(fpRate)) / (log(2)^2));`
- O número ótimo de funções hash que minimiza a taxa de falsos positivos para esse espaço alocado é calculado em Matlab como: `obj.numHashes = max(1, round((obj.numBits / expectedItems) * log(2)));`

Para mitigar o custo computacional de calcular múltiplas funções de hash independentes, implementou-se a otimização matemática de Kirsch-Mitzenmacher. Esta técnica demonstra que duas funções hash base (`h1` baseada no algoritmo DJB2 e `h2` baseada no algoritmo SDBM) são suficientes para simular as `k` posições necessárias através de uma combinação linear vetorizada. Em código Matlab, isto traduz-se de forma instantânea sem loops através

```
da linha: positions = mod(h1 + (1:obj.numHashes) .* h2, obj.numBits) + 1;
```

Esta abordagem eliminou a necessidade de gerar sementes aleatórias globais com o comando `rng`, erradicando efeitos secundários indesejados no estado global da sessão do Matlab.

2.2. Naïve Bayes (*naiveBayes.m*)

O classificador probabilístico foi desenhado para suportar duas abordagens clássicas de processamento de linguagem natural: **Multinomial** (focado na frequência absoluta das palavras) e **Bernoulli** (focado estritamente na presença ou ausência).

Na variante Bernoulli, aplicou-se o devido rigor matemático: o treino foi estruturado para contabilizar a proporção de documentos da classe que contêm a palavra, utilizando o denominador de Laplace com ajuste para dois estados (presença ou ausência). Em código Matlab, a verosimilhança positiva é dada por: `log((counts + 1) / (numClassDocs + 2));`

Adicionalmente, o algoritmo de classificação foi expandido para avaliar não apenas as palavras presentes, mas também a penalização decorrente da **ausência** de termos do vocabulário global. A probabilidade de ausência é pré-calculada no treino como: `log((numClassDocs - counts + 1) / (numClassDocs + 2));`

Durante a classificação, o sistema soma os logs das probabilidades das palavras presentes e das palavras ausentes, operando através de indexação lógica e vetorização, contornando loops sequenciais pesados.

2.3. MinHash (*minHash.m*)

Este módulo atua como um estimador estatístico do coeficiente de similaridade de Jaccard entre conjuntos textuais de dimensão elevada. O gerador de *shingles* suporta partições ao nível de caracteres (tri-shingles) ou ao nível de sequências de palavras, conferindo flexibilidade analítica ao sistema.

No cálculo da assinatura, corrigiu-se o risco de *overflow* numérico em precisão dupla (`double`) — onde a exponenciação polinomial clássica excedia o limite de inteiros seguros do Matlab (2^{53}) —, adotando-se a **Regra de Horner** com aplicação iterativa do operador `mod` sob o número primo de Mersenne ($2^{31}-1$). Em código Matlab, o hash de cada elemento é acumulado num ciclo simples através da instrução: `h = mod(h * 31 + str(b), obj.Prime);`

Para maximizar a performance na aplicação conjunta, o módulo encapsula o método matricial `computeSimilarityMatrix`. Este método avalia a colisão de hashes de centenas de documentos em bloco através de matrizes nativas do Matlab, eliminando ciclos e calculando a fração de coincidências na forma: `coincidencias = sum(Signatures(:, n1) == Signatures(:, n1+1:Nu), 1) / k_hash;`

3. INSTRUÇÕES DE EXECUÇÃO (COMO CORRER OS PROGRAMAS)

Para assegurar a portabilidade total do projeto e simplificar a avaliação por parte dos docentes, todos os scripts recorrem a caminhos relativos e mapeamento dinâmico via `addpath`.

Requisitos preliminares: O utilizador deve extrair e colocar o ficheiro de dados `news.csv` (e o `u.data`, se disponível) diretamente na pasta raiz do projeto (`Project-MPEI/`).

- **Execução dos Testes Unitários:**
 - Abrir o Matlab e navegar para o diretório raiz `Project-MPEI/`.
 - Na consola (*Command Window*), executar `test_bloomFilter` para validar as taxas de falsos positivos e a remoção no modo *counting*.
 - Executar `test_naiveBayes` para visualizar a tabela comparativa multi-classe entre as variantes Bernoulli e Multinomial.
 - Executar `test_minHash` para analisar as distâncias estimadas face ao Jaccard real e aferir a sensibilidade ao parâmetro `k`.
- **Execução da Aplicação Conjunta:**
 - Na consola do Matlab, digitar `demo_conjunta` e premir *Enter*. O terminal exibirá o log detalhado do pipeline do feed inteligente a processar dados em tempo real.

4. APRESENTAÇÃO E ANÁLISE CRÍTICA DOS RESULTADOS

4.1. Resultados do Módulo Bloom Filter

Os testes unitários provaram empiricamente que o Filtro de Bloom **não gera falsos negativos** (Falsos Negativos = 0), em total conformidade com a teoria. O teste de Falsos Positivos, conduzido com amostras independentes extraídas da segunda metade do dataset real, validou a eficácia do redimensionamento ótimo:

- Para uma taxa teórica programada de 10.00%, obteve-se uma taxa real de falsos positivos de aproximadamente 9.60%.
- Para uma taxa teórica programada de 1.00%, obteve-se uma taxa real de falsos positivos de aproximadamente 0.85%.

O modo *counting* exibiu uma taxa de sucesso de remoção de 100% nas amostras testadas, confirmando a correta gestão aritmética dos contadores face às colisões.

4.2. Resultados do Módulo Naïve Bayes

O teste multi-classe dividiu o dataset de notícias na proporção de 70% para treino e 30% para teste. O modelo **Multinomial** registou uma exatidão global superior à variante **Bernoulli** na classificação de títulos curtos de notícias. Isto justifica-se porque, embora os títulos possuam pouca repetição de termos num único documento, a contagem absoluta de ocorrências de palavras-chave ao longo de toda a classe oferece uma discriminação de verosimilhança mais expressiva do que a mera contagem binária de presença/ausência de

documentos realizada por Bernoulli. Os tempos de execução mantiveram-se na ordem dos centésimos de segundo devido à eliminação de loops internos.

4.3. Resultados do Módulo MinHash

A validação estatística confirmou a convergência do estimador de Jaccard. À medida que o número de funções de hash (parâmetro k) foi expandido de $k = 50$ para $k = 150$, o erro absoluto médio da aproximação decresceu sistematicamente de aproximadamente 0.052 para 0.031, validando a propriedade teórica de erro limitado pela raiz do inverso de k . A nível de performance computacional, o método de comparação matricial em bloco gerou um aumento de velocidade (*speedup*) superior a 15 vezes face ao ciclo de cálculo exato de interseções de strings, legitimando o uso do MinHash para volumes elevados de informação.

5. DESCRIÇÃO DETALHADA DA APLICAÇÃO DE USO CONJUNTO

A aplicação `demo_conjunta.m` orquestra os três módulos desenvolvidos para materializar a lógica de negócio de um **Feed de Notícias Inteligente e Personalizado**. O sistema simula um cenário real em que um utilizador interage com uma aplicação de leitura no seu telemóvel, processando os fluxos textuais recebidos através do seguinte pipeline sequencial em texto:

1. **Entrada da Notícia e Pré-processamento:** O sistema intereta uma nova notícia que acaba de chegar da internet (simulada pelas strings de teste).
2. **Filtro de Controlo de Redundância Absoluta (Bloom Filter):** A string do título é submetida ao método `lookup` do Filtro de Bloom, que atua como o histórico pessoal de leitura do utilizador. Se o filtro retornar positivo (`true`), significa que o utilizador já leu essa notícia exata no passado. O sistema emite um aviso de bloqueio e descarta imediatamente o artigo com custo constante, poupando recursos e evitando a execução dos algoritmos subsequentes. Se retornar negativo (`false`), o artigo é considerado inédito e avança no pipeline.
3. **Triagem Temática e Classificação (Naïve Bayes):** A notícia inédita é entregue ao classificador Naïve Bayes (treinado previamente com 1500 notícias do ficheiro `news.csv`). O modelo analisa estatisticamente o vocabulário do título e carimba a categoria temática correspondente (Mundo, Desporto, Economia ou Ciência/Tecnologia), definindo em que secção personalizada do feed o artigo deve ser depositado.
4. **Análise de Proximidade Semântica e Agrupamento (MinHash):** O título da notícia é convertido em tri-shingles e a sua assinatura compacta é gerada. O sistema utiliza a comparação vetorizada do MinHash para confrontar esta assinatura com as assinaturas de todas as notícias do arquivo histórico. Se a distância de Jaccard estimada for inferior ao limiar de 0.45, o sistema deteta que a notícia trata de um assunto que outros jornais já reportaram. Em vez de duplicar a informação no ecrã

do utilizador, o sistema emite um alerta e agrupa dinamicamente os artigos sob o mesmo tópico conceitual (estilo "Ver cobertura completa deste evento").

5. **Publicação e Atualização do Histórico:** Caso o artigo seja validado, ele é impresso com sucesso no feed do utilizador. No mesmo instante, a string é inserida no Filtro de Bloom (`insert`), garantindo que o histórico do utilizador é atualizado em tempo real para as sessões seguintes.

6. VANTAGENS E LIMITAÇÕES DAS SOLUÇÕES PROPOSTAS

Vantagens:

- **Eficiência Computacional de Entrada:** O posicionamento do Filtro de Bloom como primeira barreira protege o servidor, descartando repetições de forma ultra-rápida sem ativar os classificadores de texto pesados.
- **Pesquisa Semântica Vetorizada:** A eliminação de loops no cálculo do MinHash permite cruzar e comparar centenas de artigos em simultâneo através de operações nativas de matrizes do Matlab, garantindo escalabilidade para fluxos contínuos de informação.
- **Isolamento de Aleatoriedade:** A substituição do comando global `rng` pela classe local `RandStream` garantiu a reprodutibilidade dos coeficientes do MinHash sem corromper o comportamento pseudoaleatório dos restantes scripts.

Limitações:

- **Saturação e Falsos Positivos do Filtro de Bloom:** Com a inserção contínua de dezenas de milhares de notícias ao longo do tempo, o vetor de bits sofrerá saturação. Isto aumentará a probabilidade real de falsos positivos, o que pode levar o sistema a ocultar erradamente uma notícia inédita ao utilizador.
- **Premissa de Independência de Naïve Bayes:** O classificador assume de forma "ingénua" que a ocorrência de uma palavra é totalmente independente das palavras vizinhas, perdendo a riqueza contextual de expressões compostas ou ironias presentes nos títulos.
- **Rigidez do Limiar de Jaccard:** A fixação de um limiar estático (0.45) no MinHash pode falhar na identificação de artigos parentes caso órgãos de comunicação diferentes utilizem vocabulários excessivamente distintos para descrever rigorosamente o mesmo acontecimento.